

Pruebas de Software

Cuando iniciamos en el mundo del desarrollo de software, generalmente estamos muy enfocados en hacer que una característica funcione, o arreglar un bug y dedicamos tiempo en un ciclo de: pruebo manualmente lo que los usuarios me piden, funciona? no, verifico que cambios necesito, hago los cambios en el código y vuelvo a probar que el requerimiento funciona, funciona? si, lo pongo en producción.

Este ciclo de desarrollo está bien para proyectos pequeños que se van a quedar pequeños, y no críticos. El problema está en que no somos adivinos y no podemos saber a futuro que lo que estamos desarrollando va a quedarse pequeño y no crítico para el negocio.

Generalmente no pasa eso, los desarrollos se realizan por algo y mientras más se usa más crítico se vuelve. Por esta razón es importante desde los inicios de un nuevo proyecto tener una infraestructura adecuada para tener pruebas automatizadas que las puedan ejecutar los desarrolladores localmente, y también se puedan ejecutar en los sistemas de integración continua. Adicional a pruebas automáticas es muy importante mantener pruebas manuales ya que las pruebas automáticas generalmente sólo verifican lo que el desarrollador piensa que se debe probar.

¿Qué son pruebas de software?

Las pruebas de software, son procesos cíclicos que permiten a los desarrolladores verificar si un requerimiento o cambio en el sistema es correcto o no. Adicionalmente permite verificar que una versión específica del software tenga todos los requerimientos solicitados. También pueden ayudar a verificar que el software puede funcionar correctamente en diferentes medios o con diferentes integraciones, como por ejemplo en diferentes sistemas operativos, o en diferentes versiones de software base.

¿Por qué son importantes las pruebas de software?

Sin pruebas me parece que es imposible conocer si estamos haciendo algo correcto, no creo que nadie pueda simplemente cambiar código y ponerlo en producción sin siquiera verificar en su ambiente que el cambio hace lo que debe hacer.

Aún en ambientes donde no se tienen pruebas automatizadas, las pruebas manuales son la única forma de verificación de que el software funcione luego

de realizar un cambio, y son válidas pero estas no escalan con el software y su complejidad.

¿Cómo funcionan las pruebas de software?

Hay muchas formas de probar software, tenemos pruebas manuales, pruebas automáticas unitarias, de integración, aceptación, etc.

Generalmente los desarrolladores primero deciden qué comportamiento o característica requiere una validación, planifican cómo se puede verificar que el software funciona como se detalla, crea un script de prueba para verificar y ejecuta este script cada vez que hace cambios en el código hasta confirmar que funciona.

La parte de “crear un script de prueba”, es así tanto para pruebas manuales o automáticas, la diferencia radica en que para las manuales lo crea en su mente y en las automáticas lo crea en un archivo de código. Cómo pueden ir viendo las pruebas manuales cumplen el mismo papel que las automáticas pero de la forma en la que se crean no se pueden compartir con el equipo de trabajo y son muy lentas en ejecutar.

Beneficios de pruebas en el software

- ✓ **Asegura funcionalidad completa.** Asegura que todos los requerimientos desarrollados estén presentes en el producto final.
- ✓ **Alerta temprana.** Permite saber muy temprano en el desarrollo de software acerca de defectos que pueden afectar negativamente al usuario.
- ✓ **Reconstrucción de código.** Si tenemos un conjunto de pruebas muy amplio, es mucho más fácil hacer reconstrucciones internas del código para mejorar arquitectura sin afectar al usuario final.

Estos son algunos de los beneficios, debe haber muchos más. Pero lo más importante que me gustaría compartir es que las pruebas son muy críticas en el desarrollo, y que está bien si lo hace manualmente por ahora, pero que deben comenzar a planificar un cambio en sus mentes y en sus procesos para incorporar pruebas automáticas, al principio es muy duro van a pensar que escribir los tests los hace más lentos, que no son importantes, que hacen más complejo el desarrollo, pero una vez interiorizado el desarrollo con pruebas automáticas van a ver atrás y preguntarse ¿cómo realizaron tantos sistemas sin pruebas automáticas? Les va a parecer una locura. Sí tienen que cruzar ese valle oscuro, pero al otro lado hay cosas muy buenas, como tranquilidad de que no vas a dañar nada en producción, aumentar la calidad de tus productos para que el cliente ya no sea el departamento de QA.

Tipos De Pruebas de Software

Vamos a revisar los tipos de pruebas, por ejemplo dependiendo del tipo de defecto que verifican las pruebas cada especialización tiene su nombre, pero todos los tipos de pruebas en general se pueden clasificar como pruebas de Caja Negra o Caja Blanca.

Pruebas de Caja Negra

Toda la arquitectura interna del sistema es oculta a quien lo prueba. Es decir el usuario de prueba solo conoce **lo que el producto se supone que tiene que hacer** pero no **cómo lo hace**. Los usuarios de prueba solo observan los resultados o el comportamiento del producto y no necesitan ser programadores. Generalmente estas pruebas las realizan los usuarios finales o personas que no son parte del proceso de desarrollo.

Pruebas de Caja Blanca

Es lo opuesto a las anteriores, es decir que quien prueba conoce la estructura interna del software. Estos usuarios de prueba evalúan la lógica de los programas a través de casos de prueba específicos. Al auditar el flujo de las entradas de prueba, el usuario puede verificar que todos los casos han sido manejados correctamente.

Otra clasificación general para los métodos de pruebas es pruebas manuales versus pruebas automatizadas. Muchos tipos de pruebas se pueden hacer tanto manualmente como de forma automatizada. Esta distinción indica como la prueba es completada.

Pruebas Manuales

Una persona como probador toma el rol de un usuario final del software y chequea casos de prueba uno por uno. Es una forma tradicional de verificar el software y en algunos casos es necesario porque puede detectar cosas que no pueden las pruebas automatizadas como apariencia visual de un sitio. Pero esta forma de ejecutar pruebas no escala, cuando el software es muy grande y complejo no se puede volver a probar todo el sistema.

Pruebas Automatizadas

Es el proceso de usar software denominado un **framework de pruebas** para crear casos de pruebas que se ejecutan y comparan el resultado del programa con el resultado esperado. Algunos ejemplos de este tipo de frameworks son selenium, codeception, cucumber.

Ahora revisaremos las metodologías de pruebas clasificadas entre funcionales y no funcionales, la diferencia está en si la prueba se enfoca en el comportamiento del software o su operación interna.

Pruebas Funcionales

Son un tipo de **pruebas de caja negra** que generan los casos de prueba usando los requerimientos y especificaciones del software.

Algunas de estas metodologías incluyen:

- Pruebas unitarias (Unit testing)
- Pruebas de integración (Integration testing)
- Pruebas de sistema (System testing)
- Pruebas de aceptación de usuario (Acceptance testing)
- Pruebas de regresión (Regression testing)
- Pruebas de humo (Smoke testing)

Un proceso funcional común tiene 4 etapas, abriendo el alcance del test en cada paso. El proceso inicia con:

1. Pruebas unitarias

Verifica el correcto funcionamiento de un componente individual del software, en el caso de orientación a objetos puede verificarse clases individuales. Este tipo de pruebas las realiza el programador como parte de su trabajo. Y casi siempre están automatizadas para obtener resultados muy rápidamente.

2. Pruebas de integración

Estas son usadas para verificar cómo varios componentes conectados del software funcionan juntos. Esto se hace luego de verificar que cada componente funciona individualmente, luego se valida que funcionen

bien juntos. Al igual que las unitarias estas las hace el desarrollador de forma automatizada.

3. Pruebas de sistema

Prueban el producto completo, con todos los componentes ensamblados. Verifica que módulos completos funcionen unos con otros. Generalmente este tipo de pruebas las realiza un equipo de QA.

4. Pruebas de aceptación

Verifica si el producto final satisface los requerimientos originales. Estas pruebas pueden ser realizadas por usuarios internos (**pruebas alpha**) o externos (**pruebas beta**) que revisan cómo funciona el producto al usarlo.

Existen metodologías especializadas que verifican aspectos específicos de un programa que van más allá del proceso anterior. Por ejemplo las pruebas de regresión que sirven para verificar la integridad del producto luego de un cambio o upgrade verifican la salida del nuevo programa con la salida de versiones anteriores del mismo. Pruebas de humo sirven para verificar rápidamente que las funciones más esenciales de un producto sigan estables, cosas como el programa se abre, una página muestra datos.

Pruebas No Funcionales

Son un tipo de pruebas que verifican cómo un programa opera en lugar de si un comportamiento es correcto. Por ejemplo un test no funcional prueba que tan bien un programa se ejecuta con una carga alta de transacciones, o mientras funciona por un largo tiempo.

Las pruebas no funcionales más usadas son:

- Pruebas de desempeño (Performance testing)
- Pruebas de seguridad (Security testing)
- Pruebas de usabilidad (Usability testing)
- Pruebas de compatibilidad (Compatibility testing)
- Pruebas de estrés (Stress o load testing)

Pruebas de desempeño

Validan la velocidad, responsividad y fiabilidad en que un sistema cumple una función. Si una función funciona pero se demora mucho o a veces funciona y otra no, este test va a fallar y debe regresar a desarrollo.

Pruebas de seguridad

Son usadas para encontrar debilidades en la seguridad de la información, hay pruebas por ejemplo de pentesting o escáner de vulnerabilidades. Pero todas buscan que se mantenga la confidencialidad, integridad, autenticación, autorización, disponibilidad y no repudiación.

Pruebas de usabilidad

Usadas para verificar si una característica del software no produce confusión o dificultad al usuario final. Generalmente se verifica por un investigador que observa cómo un usuario final realiza las tareas en el sistema. Se pide a los usuarios hacer una tarea pero sin indicarles cómo realizarlas.

Pruebas de compatibilidad

Verifican que tan bien un software se desempeña en diferentes ambientes, por ejemplo si se tuvieran el mismo software para Mac, Linux y pc.

Pruebas de estrés

Los desarrolladores verifican hasta cuánto un sistema puede seguir trabajando aumentando la cantidad de usuarios o transacciones simultáneas. La idea es tratar de encontrar los cuellos de botella o puntos débiles que pueden hacer que un sistema en vivo colapse cuando hay muchos usuarios a la vez.

Contenido complementario:

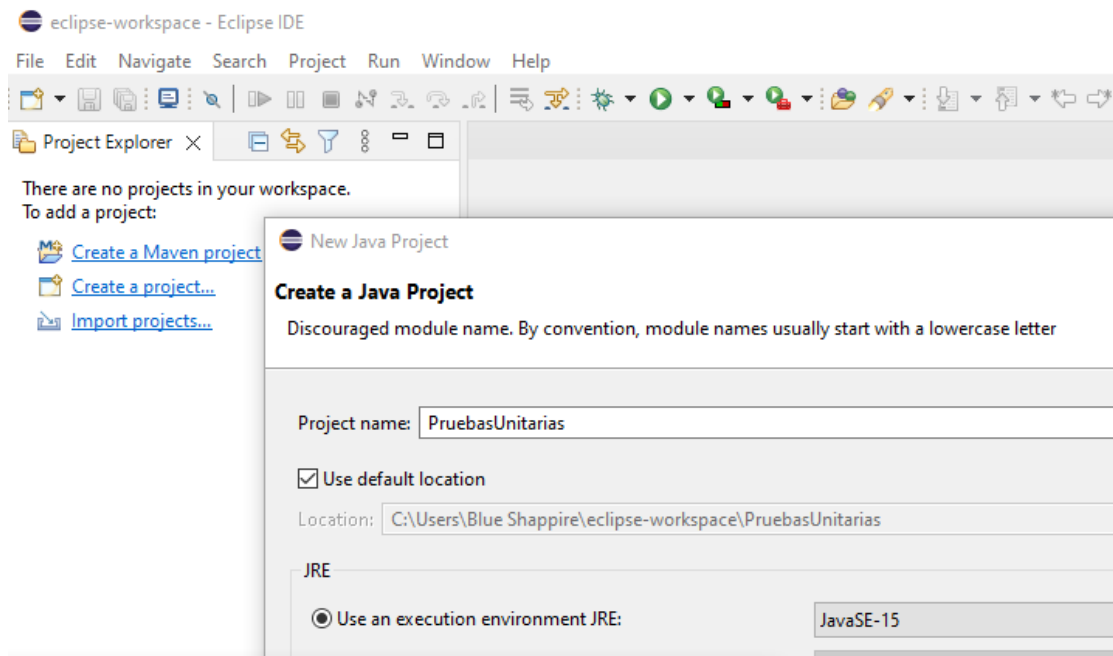
<https://jamj2000.gitbook.io/daw1-ed-apuntes/3.pruebas>

Pruebas unitarias

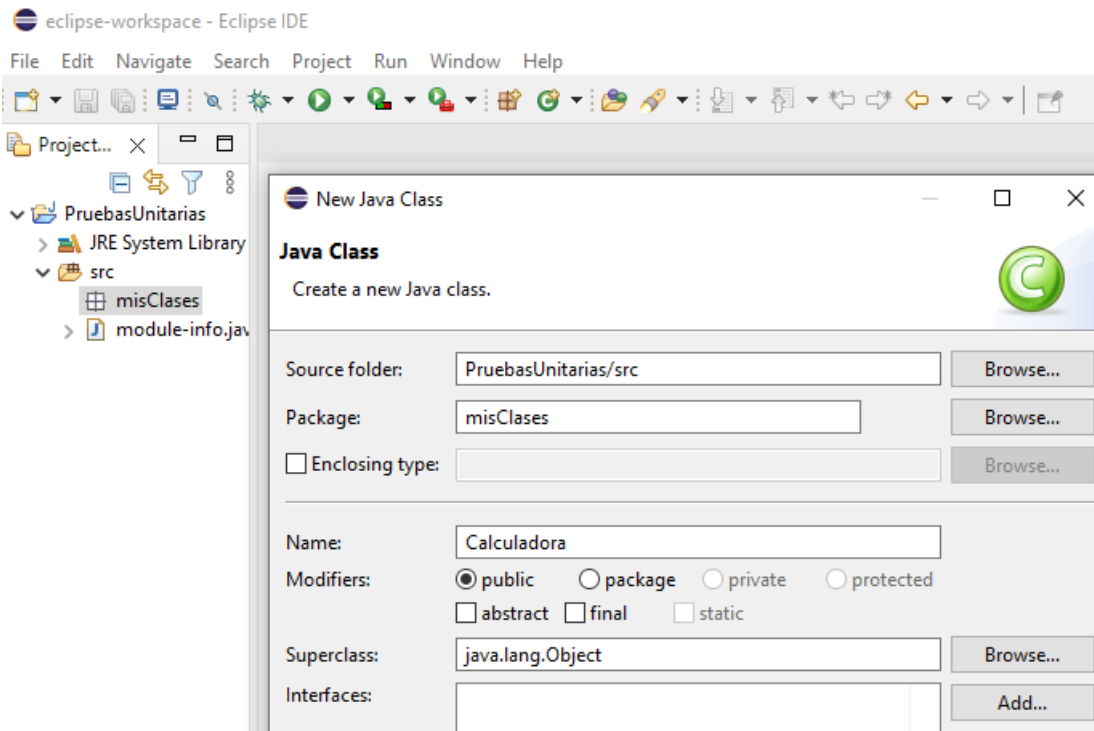
- **JUnit 4**

JUnit es un Framework Open Source para la automatización de las pruebas (tanto unitarias, como de integración) en los proyectos Software. El framework provee al usuario de herramientas, clases y métodos que le facilitan la tarea de realizar pruebas en su sistema y así asegurar su consistencia y funcionalidad.

En Eclipse desarrollamos una clase sobre la que desearemos realizar pruebas unitarias para comprobar su corrección. Por ejemplo una clase llamada Calculadora:



Creamos la clase de trabajo Calculadora en la que se crean algunos métodos básicos.



```
Calculadora.java x
1 package misClases;
2
3 public class Calculadora {
4
5     public static int suma(int a, int b) {
6
7         return a+b;
8     }
9
10    public static int resta(int a, int b) {
11
12        return a-b;
13    }
14
15    public static int multiplicacion(int a, int b) {
16
17        return a*b;
18    }
19
20    public static float division(int a, int b) {
21
22        return a/b;
23    }
24 }
```


Para comprobar que los métodos de nuestra clase funcionan correctamente podemos hacer algunas pruebas en la función principal:

```
25- public static void main(String[] args) {
26     int a=5,b=2;
27     int resultado=suma(a,b);
28
29     System.out.println(" 5 + 2 =" + resultado);
30 }
31
32 }
33
```

Console X

<terminated> Calculadora (1) [Java Application] C:\Users\Blue Shappire\.p2\pool\plugins\org

5 + 2 =7

Pero hacer las pruebas de forma manual de esta manera no es la mejor opción ya que queda código en la función principal y esto puede provocar errores, y aparte si tenemos muchas líneas de código que probar podemos perdernos fácilmente y malinterpretar resultados de las pruebas.

```
25- public static void main(String[] args) {
26     int a=5,b=2;
27     int resultado=suma(a,b);
28
29     int res_esperado=7; //Prueba manual
30     if(resultado==res_esperado) {
31         System.out.println("El método es correcto.");
32     }else {
33         System.out.println("El método no es correcto.");
34     }
35
36     System.out.println(" 5 + 2 =" + resultado);
37 }
38
39 }
40
```

Console X

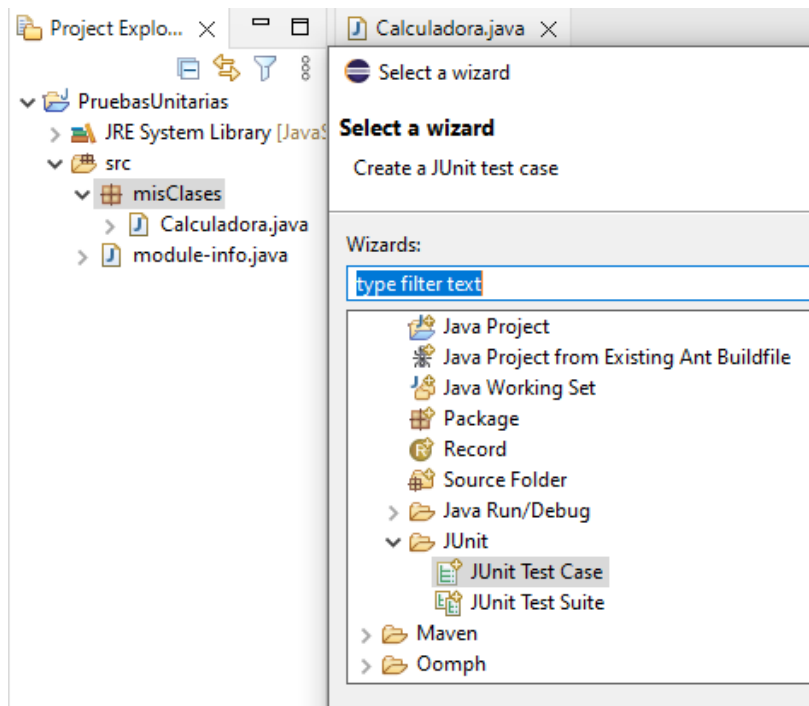
<terminated> Calculadora (1) [Java Application] C:\Users\Blue Shappire\.p2\pool\plugins\org

El método es correcto.

5 + 2 =7

Sería mucho mejor poder hacer todas las pruebas necesaria en una clase creada específicamente para ello, y hay entre en juego JUnit.

JUnit es una librería fácil de usar en Eclipse y en cualquier otro IDE. JUnit nos va a permitir crear una clase especial en la que desarrollar todas las pruebas unitarias que necesitemos sin alterar el código del proyecto.



New JUnit Test Case

Select the name of the new JUnit test case. Specify the class under test to select methods to be tested on the next page.

☐ New JUnit 3 test ☒ New JUnit 4 test ☐ New JUnit Jupiter test

Source folder:

Package:

Name:

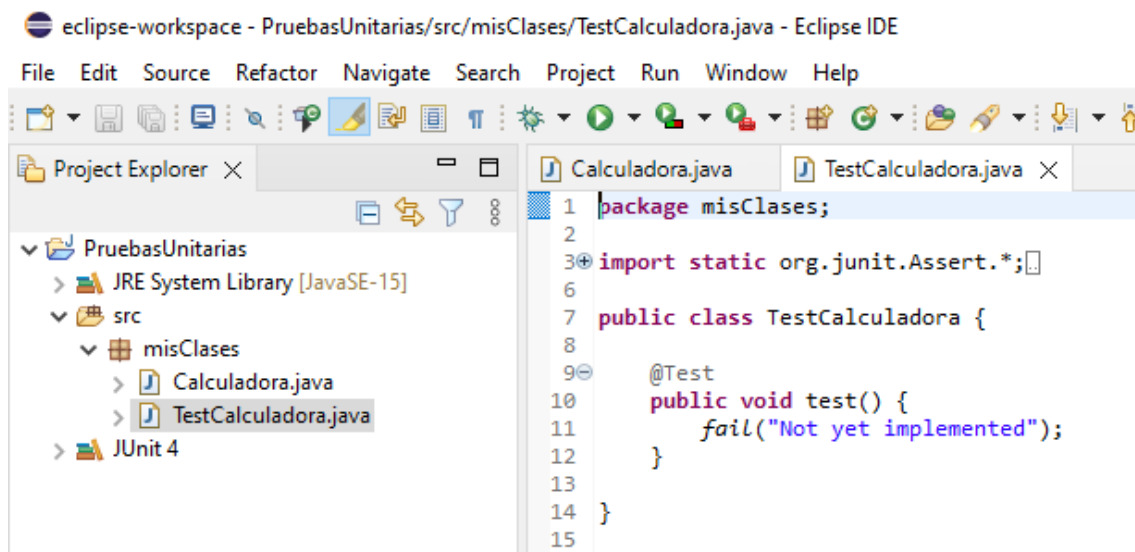
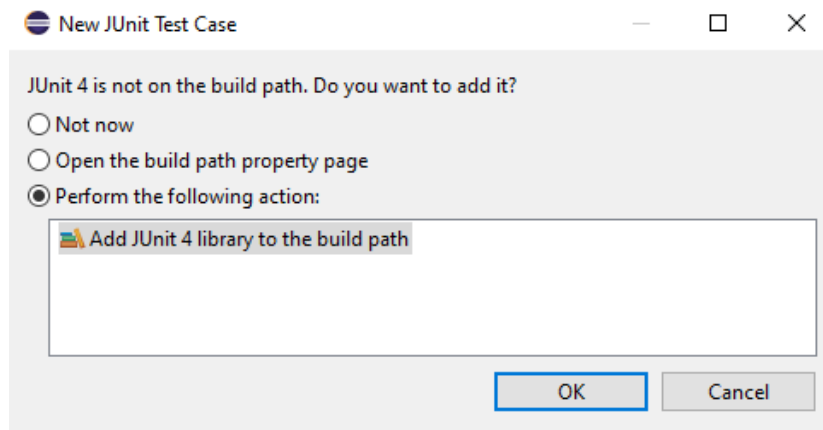
Superclass:

Which method stubs would you like to create?

☐ @BeforeClass setUpBeforeClass() ☐ @AfterClass tearDownAfterClass()
☐ @Before setUp() ☐ @After tearDown()
☐ constructor

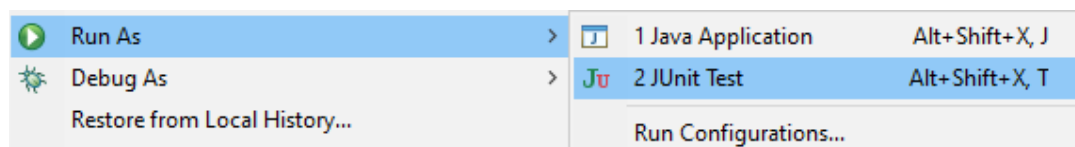
Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

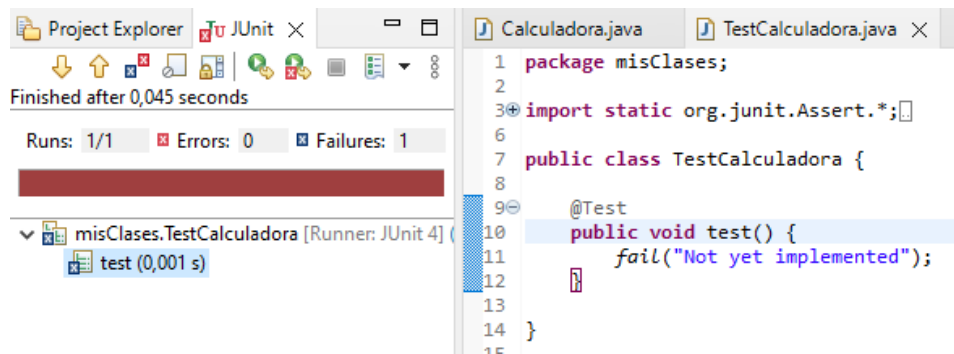
Class under test:



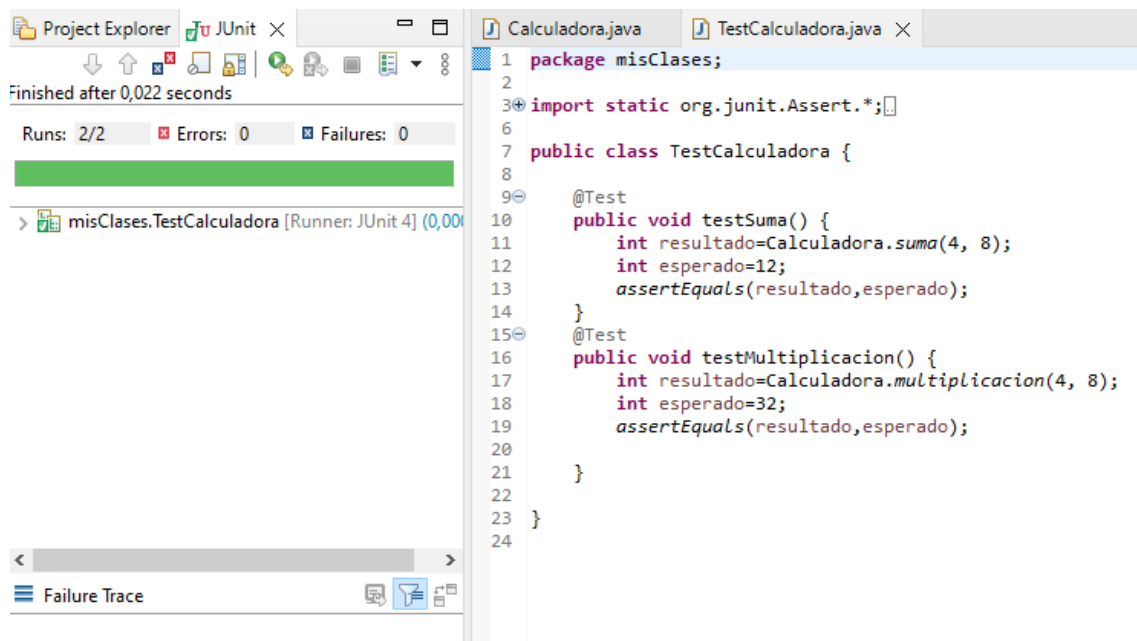
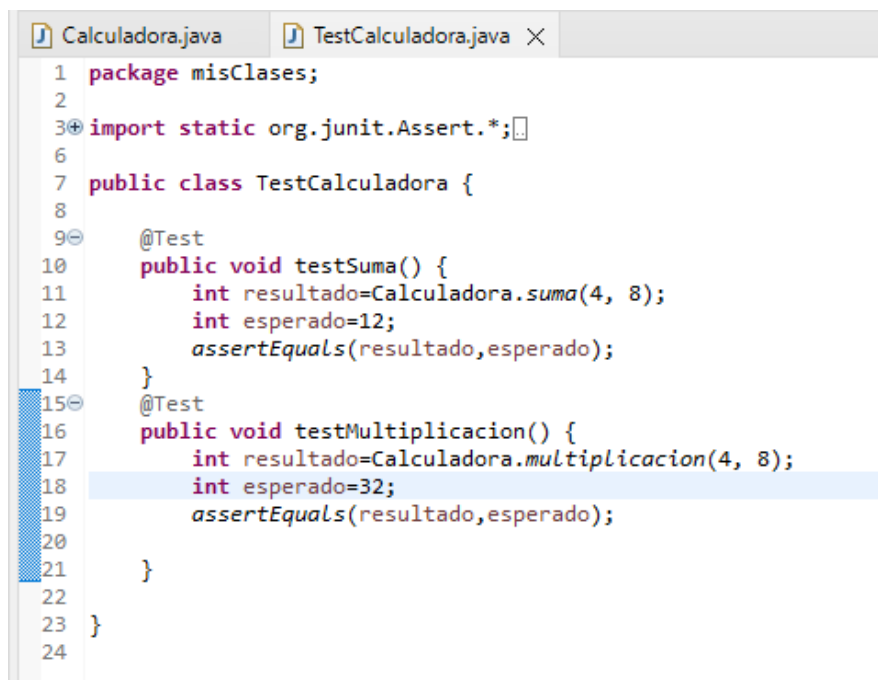
Aparece un método con la notación `@Test`. Esto lo identifica como un método de prueba. Cualquier método que tenga la notación `@Test` será reconocido por JUnit como método de prueba.

El método: `fail("Not yet implemented");` genera un fallo en la prueba. Su uso puede ser interesante en determinados casos.





Vamos a probar los métodos de la clase Calculadora:



Se han probado los dos métodos de forma simultánea en nuestra clase de prueba.

El concepto de afirmaciones (**assertions**) se atribuye a aquellas sentencias booleanas colocadas en un punto específico de un programa, las cuales serán verdaderas hasta que se demuestre lo contrario.

Este tipo de sentencias se utilizan como ayuda en las correcciones de un programa. Podemos ver la lista completa de asserts en el siguiente enlace:

<https://www.baeldung.com/junit-assertions>

Y a continuación una lista de las posibles **anotaciones** de **JUnit 4**:

- **@Test**: identifica un método como método test. Admite los parámetros:
 - **expected = <clase_excepcion.class>ón.class>** devuelve resultado fallido en caso de que no se lance la excepción indicada
 - En JUnit 5 se sustituye por `Assertions.assertThrows(<clase_excepcion.class>)`.
 - **timeout = <milisegundos>** devuelve resultado fallido en caso de que la ejecución dure más de milisegundos.
 - En JUnit 5 se sustituye por `Assertions.assertTimeout(<milisegundos>)`.
- **@Before**: se ejecuta antes de cada test. Normalmente se utiliza para preparar el entorno de testing (por ejemplo: inicialización de clases, lectura de datos de entrada....).
- **@After**: se ejecuta después de cada test. Normalmente se utiliza para limpiar el entorno de testing.
- **@BeforeClass**: se ejecuta una vez, antes de comenzar todos los tests. Estos métodos deben definirse como static para trabajar con JUnit.
- **@AfterClass**: se ejecuta una vez, cuando los tests han finalizado. Estos métodos deben definirse como static para trabajar con JUnit.
- **@Ignore**: el método de test debe ser desactivado (no se ejecuta).

Vamos a probar @Before y @After. Para ello modificaremos la clase Calculadora:

```

Calculadora.java X
1 package misClases;
2
3 public class Calculadora {
4
5     private int respuesta;
6
7     public Calculadora(){
8         respuesta=0;
9     }
10
11     public int sumar(int a, int b) {
12         respuesta=a+b;
13         return respuesta;
14     }
15
16     public int restar(int a, int b) {
17         respuesta=a-b;
18         return respuesta;
19     }
20
21     public int incrementar(int cant) {
22         respuesta+=cant;
23         return respuesta;
24     }
25
26     public int decrementar(int cant) {
27         respuesta-=cant;
28         return respuesta;
29     }
30     public int respuesta() {
31
32         return respuesta;
33     }
34
35 }

```

Podríamos probar los métodos de la clase:

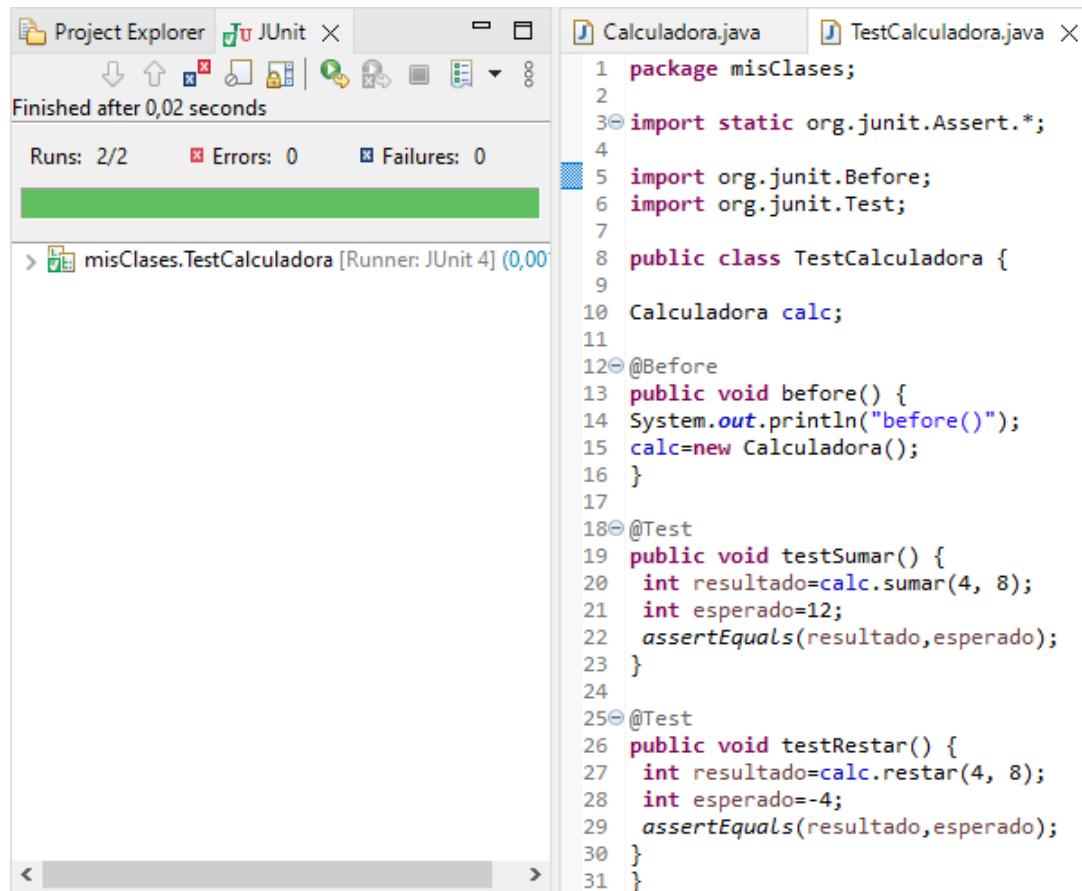
```

Project Explorer X
JUnit X
Finished after 0,02 seconds
Runs: 2/2 Errors: 0 Failures: 0
> misClases.TestCalculadora [Runner: JUnit 4] (0,00s)

Calculadora.java TestCalculadora.java X
1 package misClases;
2
3 import static org.junit.Assert.*;
4
5
6
7 public class TestCalculadora {
8
9     @Test
10     public void testSumar() {
11         Calculadora calc = new Calculadora();
12         int resultado=calc.sumar(4, 8);
13         int esperado=12;
14         assertEquals(resultado,esperado);
15     }
16
17     @Test
18     public void testRestar() {
19         Calculadora calc = new Calculadora();
20         int resultado=calc.restar(4, 8);
21         int esperado=-4;
22         assertEquals(resultado,esperado);
23     }
24 }

```

Sin embargo observamos que hay código repetido en los dos métodos. Estaría bien poder escribir estas líneas comunes una sola vez:



The screenshot shows an IDE with two tabs: `Calculadora.java` and `TestCalculadora.java`. The `TestCalculadora.java` tab is active, displaying the following code:

```
1 package misClases;
2
3 import static org.junit.Assert.*;
4
5 import org.junit.Before;
6 import org.junit.Test;
7
8 public class TestCalculadora {
9
10     Calculadora calc;
11
12     @Before
13     public void before() {
14         System.out.println("before()");
15         calc=new Calculadora();
16     }
17
18     @Test
19     public void testSumar() {
20         int resultado=calc.sumar(4, 8);
21         int esperado=12;
22         assertEquals(resultado,esperado);
23     }
24
25     @Test
26     public void testRestar() {
27         int resultado=calc.restar(4, 8);
28         int esperado=-4;
29         assertEquals(resultado,esperado);
30     }
31 }
```

On the left, the Project Explorer shows the package `misClases` containing the `TestCalculadora` class. Below it, the JUnit runner output shows: `Finished after 0,02 seconds`, `Runs: 2/2`, `Errors: 0`, and `Failures: 0`. A green progress bar indicates successful execution.

`before()` se ejecuta antes de cada método de prueba. Análogamente la anotación `@After` se ejecuta después de cada método de prueba.

```

package misClases;

import static org.junit.Assert.*;

import org.junit.After;
import org.junit.Before;
import org.junit.Test;

public class TestCalculadora {

    Calculadora calc;

    @Before
    public void before() {
        System.out.println("before()");
        calc=new Calculadora();
    }

    @After
    public void after() {
        System.out.println("after()");
    }

    @Test
    public void testSumar() {
        System.out.println("testSumar()");
        int resultado=calc.sumar(4, 8);
        int esperado=12;
        assertEquals(resultado,esperado);
    }

    @Test
    public void testRestar() {
        System.out.println("testRestar()");
        int resultado=calc.restar(4, 8);
        int esperado=-4;
        assertEquals(resultado,esperado);
    }
}

```

Da como resultado:

```

Console X
<terminated> TestCalculadora [JUnit] C:\Users\
before()
testSumar()
after()
before()
testRestar()
after()

```

Los métodos con las anotaciones **@BeforeClass** y **@AfterClass** se ejecutan solamente una vez, antes del conjunto de test y después del conjunto de test respectivamente:


```
Calculadora.java TestCalculadora.java X
4
5 import org.junit.After;
6 import org.junit.AfterClass;
7 import org.junit.Before;
8 import org.junit.BeforeClass;
9 import org.junit.Test;
10
11 public class TestCalculadora {
12
13     static Calculadora calc;
14     @BeforeClass
15     public static void beforeClass() {
16         System.out.println("beforeClass()");
17         calc=new Calculadora();
18     }
19     @AfterClass
20     public static void afterClass() {
21         System.out.println("afterClass()");
22         calc=null;
23     }
24
25     @Before
26     public void before() {
27         System.out.println("before()");
28     }
29
30
31     @After
32     public void after() {
33         System.out.println("after()");
34     }
35 }
```

Cuidado: Los métodos con la anotación `@BeforeClass` y `@AfterClass` deben ser declarados **static**.

```
Console X
<terminated> TestCalculadora [JUnit] C:\User
beforeClass()
before()
testSumar()
after()
before()
testRestar()
after()
afterClass()
```

Por otra parte vamos ahora a ver que la notación **@Test** tiene parámetros, concretamente dos: expected y timeout.

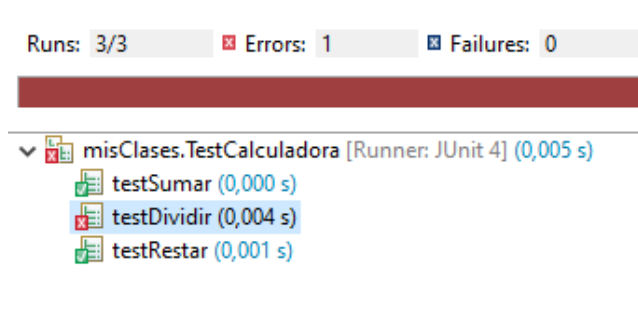
Expected es un parámetro que puede usarse para indicar que se espera de un método que lance una determinada excepción. Si no lo hace la prueba fallara. Por ejemplo, fijémonos en el método que hace divisiones:

```
public int dividir(int a, int b) {  
    respuesta=a/b;  
    return respuesta;  
}
```

Con la prueba:

```
@Test  
public void testDividir() {  
    int resultado=calc.dividir(12,0);  
}
```

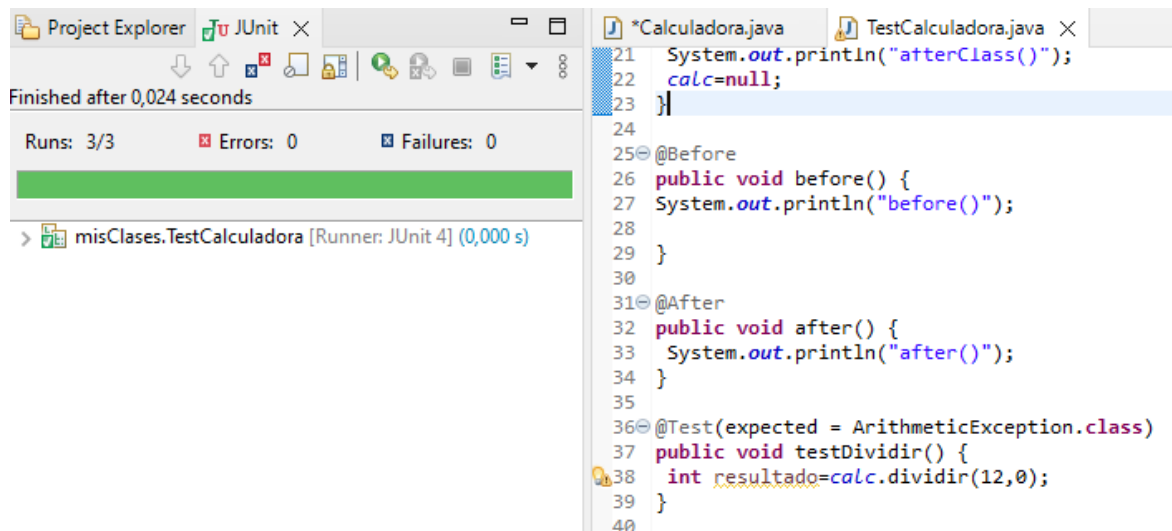
¿Pero qué pasa si dividimos entre 0? Obviamente esta situación debe producir una excepción:



Esta situación se puede controlar con el parámetro expected:

```
@Test(expected = ArithmeticException.class)  
public void testDividir() {  
    int resultado=calc.dividir(12,0);  
}
```

Cuidado con la notación, hay que poner .class al final. Si ahora ejecutamos las pruebas:



El otro parámetro, **timeout**, nos permite controlar si el método tarda en ejecutarse más tiempo de un tiempo límite que marcamos. Nos puede servir por ejemplo para detectar bucles infinitos. Por ejemplo: **@Test(timeout=100)**, lanzará fallo si la ejecución del método tarda más de 100 **milisegundos**.

Pruebas con parámetros

Hasta ahora hemos creado métodos de prueba que sirven para probar un conjunto de parámetros pasados:

```
@Test
public void testSumar() {
    System.out.println("testSumar()");
    int resultado=calc.sumar(4, 8);
    int esperado=12;
    assertEquals(resultado,esperado);
}
```

Sería interesante poder probar distintos conjuntos de parámetros de una sola vez. JUnit posee un mecanismo para hacerlo. Definamos una clase de pruebas adicional, TestParametrosCalculadora. En ella creamos unas variables de instancia con los parámetros necesarios para la prueba y un constructor que las tome como argumento. También creamos el método de prueba.

```

public class TestParametrosCalculadora {

    private int a,b,esperado;

    public TestParametrosCalculadora(int a,int b,int esperado) {
        this.a=a;
        this.b=b;
        this.esperado=esperado;
    }

    @Test
    public void testSumar() {
        Calculadora calc = new Calculadora();
        int resultado=calc.sumar(a,b);
        assertEquals(esperado,resultado);
    }

}

```

JUnit llamará el mismo al constructor que hemos creado. Para conseguir nuestro objetivo necesitamos añadir una notación a la clase:

```

@RunWith(value=Parameterized.class)
public class TestParametrosCalculadora {

```

Parameterized.class es la clase que se encarga de realizar la pruebas parametrizadas. Esta anotación nos va a permitir definir un método particular que tendrá que ser **public** y **static**, y que devolverá un objeto de tipo Iterable (por ejemplo un ArrayList). Este método tiene que tener la notación

@Parameters:

```

@RunWith(value=Parameterized.class)
public class TestParametrosCalculadora {

    @Parameters
    public static Iterable<Object[]> getData(){
        List <Object[]> obj = new ArrayList<>();
        obj.add(new Object[] {3,2,5});
        obj.add(new Object[] {1,2,3});
        obj.add(new Object[] {7,2,9});
        return obj;
    }
}

```

JUnit se encarga de todo. Al ejecutar la prueba se encarga de la inicialización, de la instanciación, del paso de variables,...

Como hemos puesto la anotación **@RunWith(value=Parameterized.class)**, al ejecutar la clase de pruebas JUnit buscará el método que tenga la anotación **@Parameters**, y va a extraer de él los datos. El Iterable devuelto contiene un array de objetos, siendo cada uno de estos objetos un array con tantas

posiciones como parámetros se necesiten para cada prueba. Si el número de parámetros no coincide, no funcionará, ya que son los parámetros con los que JUnit debe instanciar cada vez a la clase de prueba. Todo junto:

```

1 package misClases;
2
3 import static org.junit.Assert.*;
4 import java.util.ArrayList;
5 import java.util.List;
6 import org.junit.Test;
7 import org.junit.runner.RunWith;
8 import org.junit.runners.Parameterized;
9 import org.junit.runners.Parameterized.Parameters;
10
11 @RunWith(value=Parameterized.class)
12 public class TestParametrosCalculadora {
13
14     @Parameters
15     public static Iterable<Object[]> getData(){
16         List<Object[]> obj = new ArrayList<>();
17         obj.add(new Object[] {3,2,5});
18         obj.add(new Object[] {1,2,3});
19         obj.add(new Object[] {7,2,9});
20         return obj;
21     }
22
23     private int a,b,esperado;
24
25     public TestParametrosCalculadora(int a,int b,int esperado)
26     {
27         this.a=a;
28         this.b=b;
29         this.esperado=esperado;
30     }
31
32     @Test
33     public void testSumar() {
34         Calculadora calc = new Calculadora();
35         int resultado=calc.sumar(a,b);
36         assertEquals(esperado,resultado);
37     }
38
39 }

```

Si cambiamos un valor y provocamos un fallo:

```

7 import org.junit.runner.RunWith;
8 import org.junit.runners.Parameterized;
9 import org.junit.runners.Parameterized.Parameters;
10
11 @RunWith(value=Parameterized.class)
12 public class TestParametrosCalculadora {
13
14     @Parameters
15     public static Iterable<Object[]> getData(){
16         List<Object[]> obj = new ArrayList<>();
17         obj.add(new Object[] {3,2,5});
18         obj.add(new Object[] {1,2,3});
19         obj.add(new Object[] {7,2,4});
20         return obj;
21     }
22

```

- **JUnit 5**

<https://junit.org/junit5/>

<https://www.baeldung.com/junit-5>

<https://www.youtube.com/watch?v=74sCIDEYSQ4>